

文件和数据格式化

主要内容

01 文件的使用

02 一维、二维数据与 CSV 格式

03 嵌套结构数据与 JSON 格式

04 CSV 与 JSON 格式互相转换



1. 文件的使用

引入问题

- **程序结束后，数据去了哪里？**
  - 编写一个选课登记程序：依次输入学生姓名和选修课程，输入结束后，显示全部选课记录。程序用一个列表保存录入的记录

```
records = []
while True:
    name = input("请输入姓名（直接回车结束）: ")
    if name == "":
        break
    course = input("请输入选修课程: ")
    records.append([name, course])
print("当前共有{}条选课记录: ".format(len(records)))
for record in records:
    print("{}: {}".format(record[0], record[1]))
```

引入问题

- **程序结束后，数据去了哪里？**
  - 编写一个选课登记程序：依次输入学生姓名和选修课程，输入结束后，显示全部选课记录。程序用一个列表保存录入的记录
  - 第一次运行（录入张三、李四）

```
请输入姓名（直接回车结束）：张三
请输入选修课程：Python程序设计
请输入姓名（直接回车结束）：李四
请输入选修课程：大学英语
请输入姓名（直接回车结束）：
当前共有2条选课记录：
张三：Python程序设计
李四：大学英语
```

引入问题

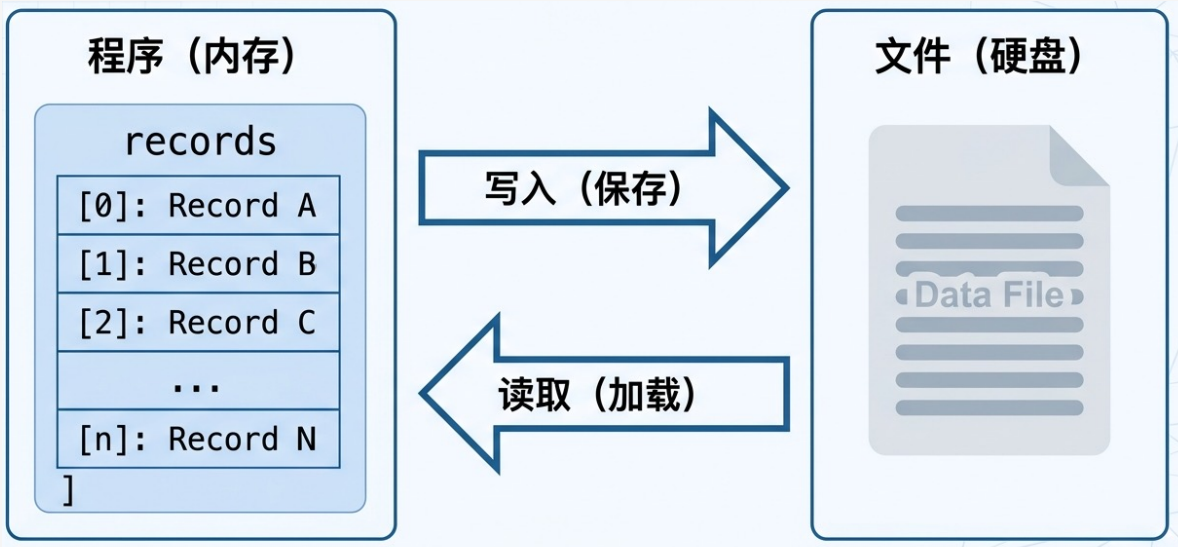
- **程序结束后，数据去了哪里？**
  - 编写一个选课登记程序：依次输入学生姓名和选修课程，输入结束后，显示全部选课记录。程序用一个列表保存录入的记录
  - 第二次运行（录入王五）

```
请输入姓名（直接回车结束）：王五
请输入选修课程：高等数学
请输入姓名（直接回车结束）：
当前共有1条选课记录：
王五：高等数学
```

- 第二次运行时，第一次录入的张三、李四两条记录已经不存在
- 原因在于程序中的变量（如列表 records）保存在内存中。程序运行结束后，它所占用的内存会被释放

引入问题

- **程序结束后，数据去了哪里？**
  - 要在程序结束后继续保留数据，必须把数据保存到硬盘等外部存储器上的文件中，下次运行时再从文件中读取。这一过程称为数据持久化



引入问题

- **程序结束后，数据去了哪里？**
  - 要在程序结束后继续保留数据，必须把数据保存到硬盘等外部存储器上的文件中，下次运行时再从文件中读取。这一过程称为数据持久化

对比项	内存中的变量	外部存储器上的文件
程序结束后数据是否保留	不保留	保留，直到文件被修改或删除
访问方式	通过变量名直接使用	先打开文件，再读取或写入

文件与路径

- **文件**
  - 文件是存储在外部存储器（如硬盘、U 盘）上、具有名称的一组数据
  - 文件名通常由主文件名和扩展名组成，例如 records.txt 和选课登记.py
  - 扩展名用来表示文件的类型
- **路径**
  - 访问一个文件需指明它所在的位置，这一位置称为路径。路径有两种写法：
    - 绝对路径：从盘符或根目录开始的完整路径，与程序在何处运行无关，例如 D:\course\data\records.txt
    - 相对路径：相对于程序运行时的当前工作目录的路径例如 data\records.txt (当前工作目录为D:\course\)

文件的打开与关闭

操作流程

- 在 Python 中，对文件的操作都遵循“打开—读写操作—关闭”三个步骤

打开文件

- 内置函数 open() 用于打开文件。打开成功后，它返回一个文件对象
- 语法格式：
  - 文件对象 = open(文件路径, 打开模式, encoding=编码名称)
    - **文件路径**：字符串，可以是绝对路径，也可以是相对路径
    - **打开模式**：字符串，表示打开文件的目的，例如读取或写入。省略时默认为 "r"，即以只读方式打开文本文件。
    - **encoding**：读写文本文件时使用的字符编码，例如 "utf-8"

文件的打开与关闭

打开文件

- 示例：

```
file = open("records.txt", "r", encoding="utf-8")
print(file)
```

  - ⚠ encoding 必须以关键字参数的形式传入，不能忽略 encoding=
    - open() 的前四个参数依次为：open(file, mode=“r”, buffering=-1, encoding=None, ...) 其中，第三个位置参数是 buffering（缓冲区大小），要求为整数；encoding 是第四个参数
    - buffering=-1 表示由 Python 自动选择合适的缓冲方式，无需修改

文件的打开与关闭

关闭文件

- 调用文件对象的 close() 方法关闭文件。关闭文件后，程序不再占用该文件
- 对于写入操作，只有关闭文件后，写入的内容才能确保完整保存到文件中
- 语法格式：
  - 文件对象.close()

文件的打开与关闭

关闭文件

- 示例代码：打开文件 records.txt，分别在关闭前后查看文件对象的 closed 属性

```
file = open("records.txt", "r", encoding="utf-8")
print(file.closed)
file.close()
print(file.closed)
```

运行结果：
False
True

- 文件对象的 closed 属性表示文件是否已经关闭
- 调用 close() 之前，closed 的值为 False；调用之后，closed 的值为 True

打开模式（一）：r、w、x、a

基本模式

模式	用途	文件不存在时	文件已存在时
`"r"`	只读（默认模式）	引发 `FileNotFoundError`	正常打开
`"w"`	只写	创建新文件	正常打开，并清空原有内容
`"x"`	只写，仅用于新建文件	创建新文件	引发 `FileExistsError`
`"a"`	追加写入	创建新文件	正常打开，保留原有内容

打开模式（一）：r、w、x、a

- **示例一：以 "r" 模式打开不存在的文件**

```
file = open("new_records.txt", "r", encoding="utf-8")
```

运行结果：
FileNotFoundError: [Errno 2] No such file or directory: 'new_records.txt'

- **示例二：以 "w" 模式打开不存在的文件**

```
file = open("new_records.txt", "w", encoding="utf-8")
file.close()
file = open("new_records.txt", "r", encoding="utf-8")
print(file)
file.close()
```

先以 “w” 模式打开并关闭文件，再以 “r” 模式打开同一文件，没有报错，说明 "w" 模式已经创建了 new_records.txt

打开模式（一）：r、w、x、a

- **示例三：以 "x" 模式打开已存在的文件**

```
file = open("new_records.txt", "x", encoding="utf-8")
```

运行结果：
FileExistsError: [Errno 17] File exists: 'new_records.txt'

"x" 模式只能用于新建文件。文件已存在时，引发 FileExistsError
当需要保护已有文件、避免误将其清空时，应选用 "x" 模式，而不是 "w" 模式

打开模式（一）：r、w、x、a

- **示例四：以 "a" 模式打开不存在的文件**
  - 删除 new_records.txt 后，以 "a" 模式打开该文件，再以 "r" 模式检验

```
file = open("new_records.txt", "a", encoding="utf-8")
file.close()
file = open("new_records.txt", "r", encoding="utf-8")
print(file)
file.close()
```

运行结果：
<_io.TextIOWrapper name='new_records.txt' mode='r' encoding='utf-8'>

"w" 与 "a" 的区别体现在文件已存在时：前者清空原有内容，后者在原有内容的末尾继续写入

打开模式（二）：t、b、+ 的组合规则

附加字符

- 附加字符用来指定文件的类型，或增加读写功能：

附加字符	含义	说明
`t`	文本模式	默认值。`"r"` 与 `"rt"` 等价
`b`	二进制模式	用于图片、音频等非文本文件
`+`	同时可读可写	在基本模式原有功能的基础上，增加另一种读写功能

打开模式（二）：t、b、+ 的组合规则

- **附加字符**
  - 组合规则：
    - 必须有且只有一个基本模式（r、w、x、a 之一），附加字符不能单独使用
    - t 与 b 不能同时出现
    - 加上 + 后，基本模式在文件存在与否时的处理方式保持不变。例如，"w+" 仍会清空原有内容，"r+" 在文件不存在时仍会引发 FileNotFoundError

打开模式（二）：t、b、+ 的组合规则

常用组合

模式	含义
`"r"`（即 `"rt"`）	以只读方式打开文本文件
`"rb"`	以只读方式打开二进制文件
`"wb"`	以只写方式打开二进制文件
`"r+"`	以读写方式打开文本文件，文件须已存在
`"w+"`	以读写方式打开文本文件，原有内容被清空
`"a+"`	以读写方式打开文本文件，写入内容追加到末尾

打开模式（二）：t、b、+ 的组合规则

- **示例一：符合规则的组合**
  - 沿用已有文件 records.txt，分别以 "rt"、"r+"、"rb" 三种模式打开该文件

```
file = open("records.txt", "rt", encoding="utf-8")
print(file)
file.close()
file = open("records.txt", "r+", encoding="utf-8")
print(file)
file.close()
file = open("records.txt", "rb")
print(file)
file.close()
```

⚠ 以 "rb" 模式打开时，得到的是二进制文件对象（BufferedReader），对象中不包含编码信息

运行结果：

```
<_io.TextIOWrapper name='records.txt' mode='rt'
encoding='utf-8’ >
<_io.TextIOWrapper name='records.txt' mode='r+'
encoding='utf-8’ >
<_io.BufferedReader name='records.txt'>
```

打开模式（二）：t、b、+ 的组合规则

- **示例二：违反组合规则**

```
file = open("records.txt", "b")
```

运行结果：
ValueError: Must have exactly one of
create/read/write/append mode and at most one plus

只写了附加字符 b，缺少基本模式，违反规则 1

```
file = open("records.txt", "rbt")
```

运行结果：
ValueError: can't have text and binary mode at once

同时使用了 t 和 b，违反规则 2

打开模式（二）：t、b、+ 的组合规则

- **示例二：违反组合规则**

```
file = open("records.txt", "rb", encoding="utf-8")
```

运行结果：
ValueError: binary mode doesn't take an encoding argument

二进制模式不涉及字符编码，因此不能指定 encoding 参数

读取文件

读取方法

- 以读模式打开文本文件后，可以通过文件对象的方法读取文件内容

方法	语法格式	功能	返回值
`read()`	`文件对象.read(size)`	从当前位置读取 `size` 个字符；省略 `size` 时，读到文件末尾	字符串
`readline()`	`文件对象.readline()`	从当前位置读取一行	字符串（含行末的换行符 `\n`）
`readlines()`	`文件对象.readlines()`	从当前位置读取所有行	列表，每个元素为一行字符串（含 `\n`）

读取文件

示例一：read()

- 假设 records.txt 文件内容为：

张三,Python程序设计

李四,大学英语

王五,高等数学

```
file = open("records.txt", "r", encoding="utf-8")
first = file.read(2)
rest = file.read()
print(first)
print(rest)
file.close()
```

运行结果：

张三

,Python程序设计 李四,大学英语 王五,高等数学

⚠ read(2) 读取 2 个字符，一个汉字算一个字符

⚠ 第二次调用 read() 时，从“张三”之后继续读取，直到文件末尾

读取文件

示例二：readline()

```
file = open("records.txt", "r", encoding="utf-8")
line1 = file.readline()
line2 = file.readline()
print(line1)
print(line2)
print(len(line1))
file.close()
```

运行结果：

```
张三,Python程序设计

李四,大学英语

14
```

- 每次调用 readline() 读取一行，连续调用时依次读取下一行
- “张三,Python程序设计”共 13 个字符，而 len(line1) 的结果为 14，说明读取到的行包含末尾的换行符 \n

读取文件

示例三：readlines()

```
file = open("records.txt", "r", encoding="utf-8")
lines = file.readlines()
print(lines)
file.close()
```

运行结果：

['张三,Python程序设计\n', '李四,大学英语\n', '王五,高等数学\n']

- readlines() 返回一个列表，列表中的每个元素对应文件中的一行
- 每个元素的末尾都带有换行符 \n

读取文件

- **示例四：用 for 循环逐行读取**
  - for 循环有两种逐行读取的写法：
    - 遍历 readlines() 返回的列表
    - 直接遍历文件对象
  - 语法格式：

```
for 变量 in 文件对象.readlines():
    循环体

for 变量 in 文件对象:
    循环体
```

读取文件

- **写法一：遍历 readlines() 返回的列表**

```
file = open("records.txt", "r", encoding="utf-8")
for line in file.readlines():
    print(line.strip())
file.close()
```

运行结果：

张三,Python程序设计
李四,大学英语
王五,高等数学

读取文件

写法二：直接遍历文件对象

```
file = open("records.txt", "r", encoding="utf-8")
for line in file:
    print(line.strip())
file.close()
```

运行结果：
张三,Python程序设计
李四,大学英语
王五,高等数学

- 遍历文件对象时，每次循环得到一行字符串，该字符串同样包含末尾的 \n
- 用字符串的 strip() 方法去除行末的换行符后再输出

读取文件

示例四：用 for 循环逐行读取

- for 循环有两种逐行读取的写法：
  - 遍历 readlines() 返回的列表
  - 直接遍历文件对象
- 两种写法的比较：

对比项	遍历 `readlines()` 的结果	直接遍历文件对象
读取时机	循环开始前读取全部行	每次循环读取一行
内存占用	与文件大小成正比	每次只占用一行
适用情形	文件较小	文件较小或较大均适用，处理大文件时应选用此写法

写入文件

写入方法

- 以写模式（"w"、"x"、"a"，或带有 + 的模式）打开文本文件后，可以通过文件对象的方法向文件写入内容

方法	语法格式	功能	返回值
`write()`	`文件对象.write(字符串)`	将一个字符串写入文件	写入的字符个数
`writelines()`	`文件对象.writelines(列表)`	将列表中的各个字符串依次写入文件	无

⚠ 这两种方法都不会自动添加换行符。如果需要分行，必须在字符串中结尾写上换行符 \n

⚠ write() 的参数必须是字符串

```
file = open("records.txt", "w", encoding="utf-8")
file.write(3)
```

报错 TypeError: write() argument must be str, not int

写入文件

示例一：用 write() 写入时未添加换行符

```
file = open("records.txt", "w", encoding="utf-8")
file.write("张三,Python程序设计")
file.write("李四,大学英语")
file.close()
file = open("records.txt", "r", encoding="utf-8")
print(file.read())
file.close()
```

运行结果：

张三,Python程序设计李四,大学英语

⚠ 两次调用 write() 写入的内容连在了一起，文件中只有一行

写入文件

- **示例二：用 write() 写入并添加换行符**

```
file = open("records.txt", "w", encoding="utf-8")
count = file.write("张三,Python程序设计\n")
file.write("李四,大学英语\n")
file.write("王五,高等数学\n")
file.close()
print(count)
file = open("records.txt", "r", encoding="utf-8")
for line in file:
    print(line.strip())
file.close()
```

运行结果：

14
张三,Python程序设计
李四,大学英语
王五,高等数学

写入文件

示例三：用 writelines() 写入列表

```
records = ["张三,Python程序设计", "李四,大学英语", "王五,高等数学"]
file = open("records.txt", "w", encoding="utf-8")
file.writelines(records)
file.close()
file = open("records.txt", "r", encoding="utf-8")
print(file.read())
file.close()
```

运行结果：
张三,Python程序设计李四,大学英语王五,高等数学

⚠ writelines() 只是将列表中的字符串依次写入，不会在元素之间添加换行符

写入文件

- 示例三：用 writelines() 写入列表

```
records = ["张三,Python程序设计\n", "李四,大学英语\n", "王五,高等数学\n"]
file = open("records.txt", "w", encoding="utf-8")
file.writelines(records)
file.close()
file = open("records.txt", "r", encoding="utf-8")
for line in file:
    print(line.strip())
file.close()
```

在每个元素末尾加上 \n

运行结果：

张三,Python程序设计
李四,大学英语
王五,高等数学

写入文件

- **示例四：“a” 模式与 “w” 模式的区别**
  - 此时 records.txt 中已有示例三写入的三条记录。张三增选了一门课程，以 “a” 模式在文件末尾追加这条记录：

```
file = open("records.txt", "a", encoding="utf-8")
file.write("张三,线性代数\n")
file.close()
file = open("records.txt", "r", encoding="utf-8")
for line in file:
    print(line.strip())
file.close()
```

运行结果：

张三,Python程序设计
李四,大学英语
王五,高等数学
张三,线性代数

写入文件

- **示例四："a" 模式与 "w" 模式的区别**
  - 接着，再以 "w" 模式写入一条记录：

```
file = open("records.txt", "w", encoding="utf-8")
file.write("李四,数据结构\n")
file.close()
file = open("records.txt", "r", encoding="utf-8")
for line in file:
    print(line.strip())
file.close()
```

运行结果：
李四,数据结构

- 以 "a" 模式打开已存在的文件时，原有内容保留，新内容写在文件末尾
- 以 "w" 模式打开已存在的文件时，原有内容被清空，文件中只剩下新写入的内容

文件的读写位置与 seek() 方法

读写位置

- 以 "r"、"r+"、"w"、"w+" 模式打开文件时，读写位置在文件开头
- 以 "a"、"a+" 模式打开文件时，读写位置在文件末尾
- 读写位置已到达文件末尾时，再次读取得到的是空字符串
- ⚠️ 在同一个文件对象上先写入、再读取时，读写位置已经处在写入内容之后，直接读取将得不到刚写入的内容

文件的读写位置与 seek() 方法

- **seek() 方法**
  - 用于移动读写位置。在文本模式下，最常用的写法是 seek(0)，即将读写位置移回文件开头
  - 语法格式：
    - 文件对象.seek(0)
- **示例一：以 "w+" 模式写入后读取**

```
file = open("records.txt", "w+", encoding="utf-8")
file.write("张三,Python程序设计\n")
file.write("李四,大学英语\n")
content = file.read()
print(len(content))
file.seek(0)
print(file.read())
file.close()
```

运行结果：

```
0
张三,Python程序设计
李四,大学英语
```

文件的读写位置与 seek() 方法

- **seek() 方法**
  - 用于移动读写位置。在文本模式下，最常用的写法是 seek(0)，即将读写位置移回文件开头
  - 语法格式：
    - 文件对象.seek(0)
- **示例一：以 "w+" 模式写入后读取**

```
file = open("records.txt", "w+", encoding="utf-8")
file.write("张三,Python程序设计\n")
file.write("李四,大学英语\n")
content = file.read()
print(len(content))
file.seek(0)
print(file.read())
file.close()
```

- 写入两条记录后，读写位置位于文件末尾，此时调用 read() 得到空字符串，其长度为 0
- 调用 seek(0) 将读写位置移回文件开头，再次调用 read()，读到了刚写入的两条记录

文件的读写位置与 seek() 方法

- **示例二：以 "a+" 模式追加后读取**
  - 此时 records.txt 中已有示例一写入的两条记录

```
file = open("records.txt", "a+", encoding="utf-8")
file.write("王五,高等数学\n")
print(len(file.read()))
file.seek(0)
for line in file:
    print(line.strip())
file.close()
```

运行结果：

0
张三,Python程序设计
李四,大学英语
王五,高等数学

- 以 "a+" 模式打开文件时，读写位置在文件末尾；追加记录后，读写位置仍在文件末尾，直接读取得到空字符串
- 调用 seek(0) 后，读到了文件的全部内容(包括原有的两条记录和新追加的一条记录)

with 语句

- **问题**：发生错误时，文件可能未被关闭
  - **解决方法一**：在 finally 中关闭文件

```
file = open("records.txt", "r", encoding="utf-8")
try:
    for line in file:
        parts = line.strip().split(",")
        print("{}选修了{}".format(parts[0], parts[1]))
except IndexError:
    print("选课记录格式错误")
finally:
    file.close()
print(file.closed)
```

with 语句

- **问题**：发生错误时，文件可能未被关闭
  - **解决方法二**：使用 with 语句
    - with 语句在打开文件的同时，规定了文件的使用范围：缩进的语句块执行结束后，无论是正常结束还是因发生错误而中断，文件都会被自动关闭，无须调用 close()
    - 语法格式：

```
with open(文件路径, 打开模式, encoding=编码名称) as 文件对象:
    操作文件的语句块
```

as 之后的变量名用于接收 open() 返回的文件对象

with 语句

- **问题**：发生错误时，文件可能未被关闭
  - **解决方法二**：使用 with 语句

```
with open("records.txt", "w", encoding="utf-8") as file:
    file.write("张三,Python程序设计\n")
    file.write("李四,大学英语\n")
    print(file.closed)
print(file.closed)
```

运行结果：
False
True

- 在 with 语句块内部，文件处于打开状态，file.closed 的值为 False
- 语句块执行结束后，文件已被自动关闭，file.closed 的值为 True

with 语句

- **问题**：发生错误时，文件可能未被关闭
  - **解决方法二**：使用 with 语句

```
try:
    with open("records.txt", "r", encoding="utf-8") as file:
        for line in file:
            parts = line.strip().split(",")
            print("{}选修了{}".format(parts[0], parts[1]))
except IndexError:
    print("选课记录格式错误")
print(file.closed) 返回 True
```

- 语句块中发生错误后，文件同样被自动关闭，file.closed 的值为 True
- 与在 finally 中关闭文件相比，with 语句不需要单独写出 close()，代码更简洁

with 语句

- **问题**：发生错误时，文件可能未被关闭

对比项	手动调用 `close()`	在 `finally` 中关闭	`with` 语句
正常结束时文件是否关闭	是	是	是
发生错误时文件是否关闭	否	是	是
是否需要写出 `close()`	需要	需要	不需要

文件操作中的异常处理

文件操作中常见的异常

异常类型	引发情形
`FileNotFoundError`	以 `"r"` 或 `"r+"` 模式打开不存在的文件
`FileExistsError`	以 `"x"` 模式打开已存在的文件
`UnicodeDecodeError`	读取文件时指定的编码与文件实际采用的编码不一致

利用 try-except 语句捕获并处理

```
try:
    with open(文件路径, 打开模式, encoding=编码名称) as 文件对象:
        操作文件的语句块
except 异常类型1:
    处理异常1的语句块
except 异常类型2:
    处理异常2的语句块
```

文本文件与二进制文件

- **两类文件**
  - 按照内容的存储方式，文件可以分为文本文件和二进制文件两类：
    - 文本文件：按某种字符编码保存的文字
    - 二进制文件：图片、音频、视频、可执行程序等非文字数据
- **文本模式与二进制模式的区别**
  - 文本模式按照 encoding 指定的编码，将文件内容还原为字符串；
  - 二进制模式不进行任何转换，直接得到文件中实际保存的内容，即字节串

文本文件与二进制文件

- **示例一：分别以文本模式和二进制模式读取同一文件**
  - records.txt 以 UTF-8 编码保存，内容为一条选课记录

```
with open("records.txt", "r", encoding="utf-8") as file:
    text = file.read()
with open("records.txt", "rb") as file:
    data = file.read()
print(text)
print(data)
print(type(text))
print(type(data))
```

运行结果：

```
张三,Python程序设计
b'\xe5\xbc\xa0\xe4\xb8\x89,Python\xe7\xa8\x8
b\xe5\xba\x8f\xe8\xae\xbe\xe8\xae\xa1\n'
<class 'str'  >
<class 'bytes'>
```

- 以文本模式读取，得到字符串，内容与写入时的文字一致。
- 以二进制模式读取，得到字节串，输出时以 b 开头。其中英文字母和逗号原样显示，每个汉字则显示为若干个以 \x 开头的编码值

文本文件与二进制文件

示例二：以文本模式读取图片文件

```
with open("photo.jpg", "r", encoding="utf-8") as file:
    content = file.read()
```

运行结果：

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in position 0: invalid start byte

- 图片中保存的不是文字，无法按任何字符编码还原为字符串，因此以文本模式读取时引发 UnicodeDecodeError
- 处理图片等非文本文件时，必须使用二进制模式

练习题

- 第 1 题：写出下列程序的运行结果。若程序运行出错，写出出错前的输出，并指出错误类型

```
with open("courses.txt", "w", encoding="utf-8") as file:
    file.write("Python程序设计")
    file.writelines(["\n大学英语", "高等数学\n"])
with open("courses.txt", "r", encoding="utf-8") as file:
    print(file.read(6))
    print(file.readline())
    print(len(file.readlines()))
```

运行结果：

```
Python
程序设计

1
```

- 写入后，文件内容为 "Python程序设计\n大学英语高等数学\n"，共两行
- read(6) 从文件开头读取 6 个字符，得到 "Python”
- readline() 从当前位置读到本行末尾，得到 "程序设计\n”
- readlines() 读取剩余的全部行，只剩第二行 "大学英语高等数学\n"，列表长度为 1

练习题

- 第 2 题：写出下列程序的运行结果。若程序运行出错，写出出错前的输出，并指出错误类型

```
names = ["张三", "李四", "王五"]
for name in names:
    with open("signin.txt", "w", encoding="utf-8") as file:
        file.write(name + "\n")
with open("signin.txt", "r", encoding="utf-8") as file:
    print(file.readlines())
try:
    with open("signin.txt", "x", encoding="utf-8") as file:
        file.write("签到结束\n")
except FileExistsError:
    print("文件已存在")
```

运行结果：
['王五\n']
文件已存在

- 每次循环都以 “w” 模式重新打开文件，"w" 模式会清空原有内容
- readlines() 返回的列表中，元素包含末尾的 \n
- 此时 signin.txt 已存在，以 "x" 模式打开引发 FileExistsError

练习题

- 第 3 题：写出下列程序的运行结果。若程序运行出错，写出出错前的输出，并指出错误类型

```
with open("records.txt", "w", encoding="utf-8") as file:
    file.write("张三,Python程序设计\n")
with open("records.txt", "a+", encoding="utf-8") as file:
    file.write("李四,大学英语\n")
    print(file.readline())
    file.seek(0)
    print(file.readline().strip())
    print(len(file.readlines()))
```

运行结果：

张三,Python程序设计
1

- 以 "a+" 模式打开文件时，读写位置在文件末尾
- 此时 readline() 返回空字符串，print() 输出一个空行
- seek(0) 将读写位置移回文件开头，readline() 读到第一行，用 strip() 去除换行符
- readlines() 读取剩余的行，只剩“李四,大学英语”一行